

Hello, I am a Principal Software Architect at Google. Below is the ****Enterprise-Grade Software Development Blueprint**** for the ****NexusLab Research Management System (RMS)****.

This blueprint focuses on scalability, data integrity, and compliance, ensuring the system can handle the rigorous demands of modern scientific research environments.

■ Project Overview

****NexusLab**** is a comprehensive, cloud-native Research Lab Management System designed to centralize the lifecycle of scientific research. It streamlines lab operations by integrating resource scheduling, inventory tracking (reagents, samples, equipment), electronic lab notebooks (ELN), and grant/budget management into a single pane of glass.

The system is designed to eliminate "data silos" within research institutions, ensuring that every pipetted microliter and every hour of microscope time is logged, searchable, and auditable.

■ Business Objective

- * ****Operational Efficiency:**** Reduce administrative overhead for PIs (Principal Investigators) by 40% through automated scheduling and inventory alerts.
- * ****Compliance & Reproducibility:**** Ensure 100% auditability of experimental data to meet institutional and federal (e.g., FDA, HIPAA) standards.
- * ****Resource Optimization:**** Maximize the ROI of expensive laboratory hardware through optimized booking algorithms.
- * ****Financial Transparency:**** Real-time tracking of grant expenditures against experimental costs.

■ User Roles

- * ****System Administrator:**** Manages global settings, user onboarding, and system integrations.
- * ****Principal Investigator (PI):**** Manages lab funds, approves high-cost purchases, and oversees research projects.
- * ****Lab Manager:**** Handles day-to-day operations, inventory procurement, and equipment maintenance.
- * ****Researcher/Scientist:**** Performs experiments, logs data in the ELN, and reserves equipment.
- * ****Auditor:**** Read-only access to logs and compliance reports.

■ Functional Requirements

- * ****Inventory Management:**** Real-time tracking of chemicals, biological samples, and consumables with low-stock notifications.
- * ****Equipment Booking:**** A calendar-based reservation system for shared instruments with conflict resolution.
- * ****Electronic Lab Notebook (ELN):**** Rich-text editor with version control for documenting protocols and experimental results.
- * ****Project & Grant Tracking:**** Linking specific experiments to funding sources for financial reporting.
- * ****Sample Tracking (LIMS):**** Barcode/QR code integration for physical sample identification and storage location tracking (e.g., Freezer -> Shelf -> Box).
- * ****Security & Audit Logs:**** Immutable logs of all system changes for compliance.

■ Non-Functional Requirements

- * **High Availability:** 99.9% uptime using multi-zone deployment.
- * **Scalability:** Horizontal scaling of microservices to handle concurrent data-heavy uploads.
- * **Security:** AES-256 encryption at rest and TLS 1.3 in transit.
- * **Auditability:** Comprehensive change-data-capture (CDC) on all primary database entities.
- * **Performance:** < 200ms latency for 95% of API requests.
- * **Compliance:** Designed to be 21 CFR Part 11 and GDPR compliant.

■ Suggested Technology Stack

- * **Frontend:** React.js with TypeScript, Tailwind CSS, and TanStack Query (State Management).
- * **Backend:** Node.js with NestJS (Enterprise Framework) or Go (for high-performance compute modules).
- * **Database:** PostgreSQL (Primary Relational Data) + Redis (Caching and Session Management).
- * **Cloud Infrastructure:** Google Cloud Platform (GCP).
- * **DevOps:** Docker, Kubernetes (GKE), GitHub Actions for CI/CD, Terraform for IaC.
- * **Authentication:** Google Identity Platform or Auth0 (OIDC/SAML support).

■ Suggested Folder Structure

```text

nexuslab-monorepo/

■ apps/

■ ■ web-client/ # React/Next.js frontend

■ ■ api-gateway/ # NestJS Backend

■ ■ libs/

■ ■ common/ # Shared types and utilities

■ ■ database/ # TypeORM/Prisma schemas & migrations

■ ■ ui-components/ # Shared Design System

■ ■ infrastructure/

■ ■ terraform/ # GCP Resource definitions

■ ■ k8s/ # Kubernetes manifests

■ ■ docker/ # Dockerfiles for various environments

■ ■ scripts/ # Database seeding and CI helper scripts

```

■ REST API Endpoints

| Method | Endpoint | Purpose |

| :--- | :--- | :--- |

| `POST` | `/api/v1/auth/login` | User authentication and JWT issuance |

| `GET` | `/api/v1/inventory` | List all lab consumables and reagents |

| `POST` | `/api/v1/inventory/adjust` | Update stock levels (manual or barcode scan) |

`GET`	`/api/v1/equipment`	List all available lab machinery
`POST`	`/api/v1/bookings`	Reserve a time slot for specific equipment
`GET`	`/api/v1/projects/:id`	Fetch project details including linked grants
`POST`	`/api/v1/elN/entries`	Create a new experiment log entry
`GET`	`/api/v1/audit/logs`	Fetch system audit trails (Admin/Auditor only)

■ Database Design

Table: `users`

* `id` (UUID, PK), `email` (Unique), `password\_hash`, `role\_id`, `created\_at`.

Table: `inventory`

* `id` (UUID, PK), `item\_name`, `category` (Chemical/Biological/Consumable), `quantity`, `unit`, `expiry\_date`, `location\_id`.

Table: `equipment`

* `id` (UUID, PK), `name`, `model\_number`, `status` (Available/Maintenance/In-Use), `maintenance\_schedule`.

Table: `bookings`

* `id` (UUID, PK), `user\_id` (FK), `equipment\_id` (FK), `start\_time`, `end\_time`.

Table: `elN\_entries`

* `id` (UUID, PK), `project\_id` (FK), `author\_id` (FK), `content` (JSONB/Text), `version`, `is\_locked`.

■ Deployment Architecture

1. **Traffic Management:** Google Cloud Load Balancer (GCLB) routes traffic to the nearest region.
2. **Compute:** Google Kubernetes Engine (GKE) manages auto-scaling containers.
3. **Database:** Managed Cloud SQL (PostgreSQL) with automated backups and failover replicas.
4. **Storage:** Google Cloud Storage (GCS) for large experimental datasets and images.
5. **Caching:** Cloud Memorystore (Redis) for session data and frequent query caching.
6. **Observability:** Google Cloud Monitoring & Logging (formerly Stackdriver).

■ Development Roadmap

Week 1: Foundation & Identity

- * Setup GCP Project and IaC (Terraform).
- * Initialize Repository and CI/CD pipelines.
- * Implement Authentication (OAuth2/OpenID Connect).
- * Database schema modeling and initial migration.

Week 2: Core Inventory & Logistics

- * Develop Inventory Management modules.
- * Implement Barcode/QR code generation and scanning logic.
- * Build Location Management (Freezer/Shelf hierarchy).

Week 3: Resource Scheduling

- * Build the Equipment Booking Engine.
- * Implement conflict detection algorithms.
- * Develop the Calendar UI for researchers.

Week 4: Electronic Lab Notebook (ELN)

- * Integrate Rich Text Editor (e.g., TipTap or Slate).
- * Implement Version Control and "Locking" mechanism for experiment entries.
- * File upload integration for experimental raw data.

Week 5: Financials & Reporting

- * Grant management and budget tracking.
- * Reporting dashboard for PIs and Lab Managers.
- * Export functionality (PDF/CSV) for audit compliance.

Week 6: Quality Assurance & Launch

- * Load testing and Security Penetration Testing.
- * UAT (User Acceptance Testing) with beta research groups.
- * Final production deployment and documentation handover.

Prepared by:

Senior Principal Architect

Google Cloud - Enterprise Solutions